DOCKER-02 ASSIGNMENT

Step-by-Step Implementation Guide

Objective

1. Create a Tomcat container on port 8080 and deploy a sample application.
2. Create a Docker volume and deploy a Tomcat container on port 8081.
3. Limit an Nginx container to 500 MB of memory.
4. Create a Dockerfile using Amazon Linux as the base image and install Nginx.
5. Build the image and push it to Docker Hub.
6. Push the image to AWS ECR.

Prerequisites: Install Docker on Ubuntu EC2

Check whether Docker is installed:

```
docker --version
docker ps
```

If Docker is not installed, execute:

```
sudo apt update
sudo apt install docker.io -y
sudo systemctl enable --now docker
sudo usermod -aG docker $USER
```

Log out and reconnect to the EC2 instance, then execute:

```
docker ps
```

TASK 1: Create a Tomcat Container on Port 8080

Step 1: Pull the Tomcat image

```
docker pull tomcat:latest
```

```
root@ip-172-31-36-56:/home/ubuntu# docker pull tomcat
Using default tag: latest
latest: Pulling from library/tomcat
4f4fb700ef54: Pull complete
06ad70e463aa: Pull complete
130739ca07b1: Pull complete
e59a999a9c9e: Pull complete
5dec61325458: Pull complete
0b2769356b03: Pull complete
4e07a0f12b2c: Pull complete
f39241fdc590: Pull complete
89ac485ce7f9: Download complete
e0be5b59881d: Download complete
Digest: sha256:6974cddd46f09a10a0d4ffae6cd77f9122b3cf7de8f4b7c3305f35026accc81f
Status: Downloaded newer image for tomcat:latest
docker.io/library/tomcat:latest
```

Step 2: Create a sample application

mkdir -p ~/docker-02/app
cd ~/docker-02/app
nano index.html

Add the following HTML code:

```
<!DOCTYPE html>
<html>
<head>
    <title>Docker-02</title>
</head>
<body>
    <h1>Hello from Tomcat Container!</h1>
    <p>Docker-02 assignment completed.</p>
</body>
</html>
```

Save the file using Ctrl+O, press Enter, and then Ctrl+X.

Step 3: Run the Tomcat container

docker run -d --name tomcat-8080 -p 8080:8080 tomcat:latest

```
root@ip-172-31-36-56:~/docker-02/app# docker run -d  --name tomcat-8080  -p 8080:808
0  tomcat:latest
d01e9d49283f221cee35319dd83aaa534b82ab86695eccb113c963704dc2ef22
```

Step 4: Deploy the application

docker exec tomcat-8080 mkdir -p /usr/local/tomcat/webapps/ROOT
docker cp index.html tomcat-8080:/usr/local/tomcat/webapps/ROOT/index.html

```
root@ip-172-31-36-56:~/docker-02/app# docker exec tomcat-8080 mkdir -p /usr/local/to
mcat/webapps/ROOT
root@ip-172-31-36-56:~/docker-02/app# docker cp index.html tomcat-8080:/usr/local/to
mcat/webapps/ROOT/index.html
Successfully copied 2.05kB to tomcat-8080:/usr/local/tomcat/webapps/ROOT/index.html
root@ip-172-31-36-56:~/docker-02/app#
```

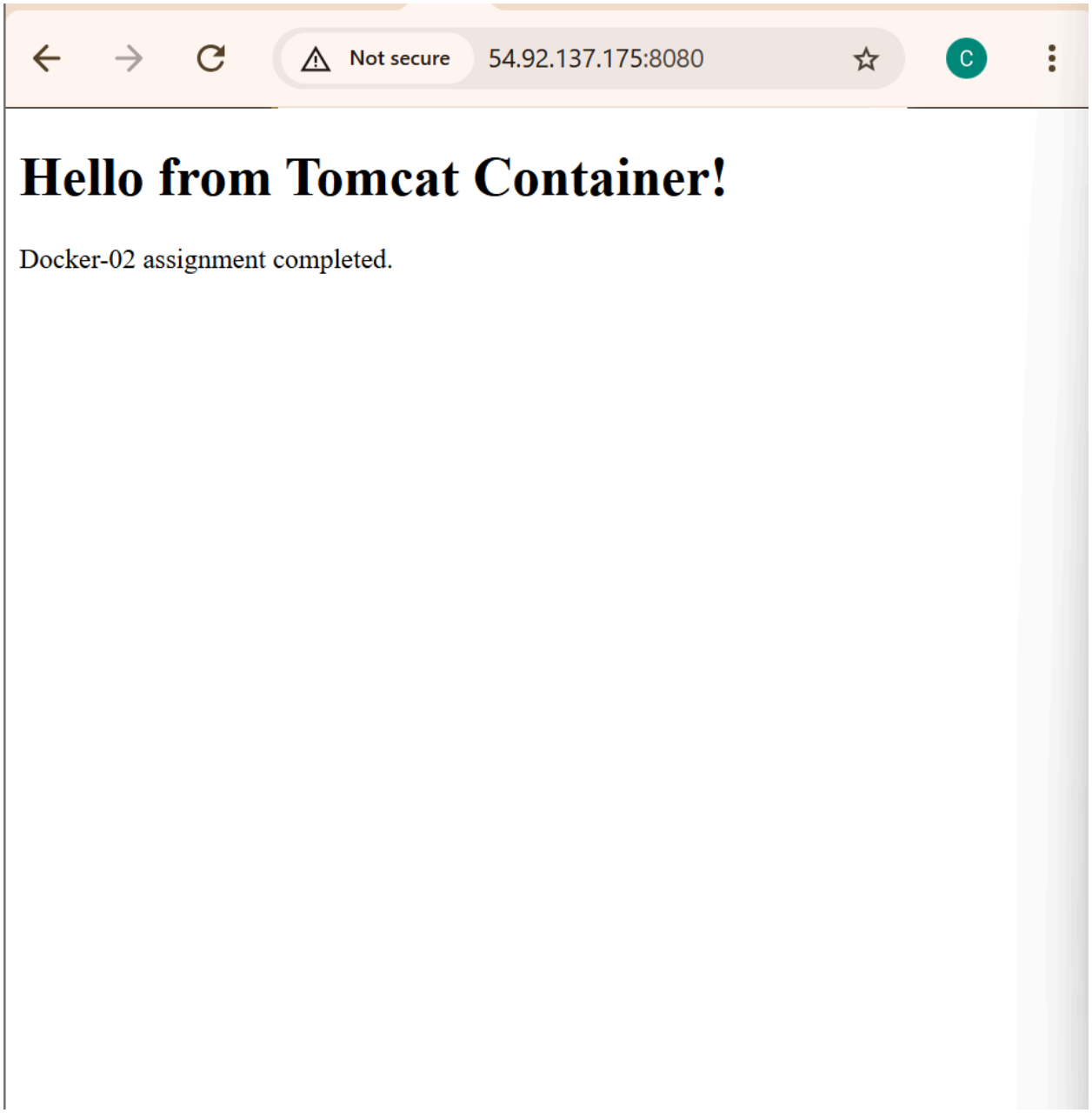
Step 5: Verify the application

Open the following URL in your browser:

http://YOUR-EC2-PUBLIC-IP:8080/

Allow inbound TCP port 8080 in the EC2 Security Group.

Expected output: Hello from Tomcat Container!



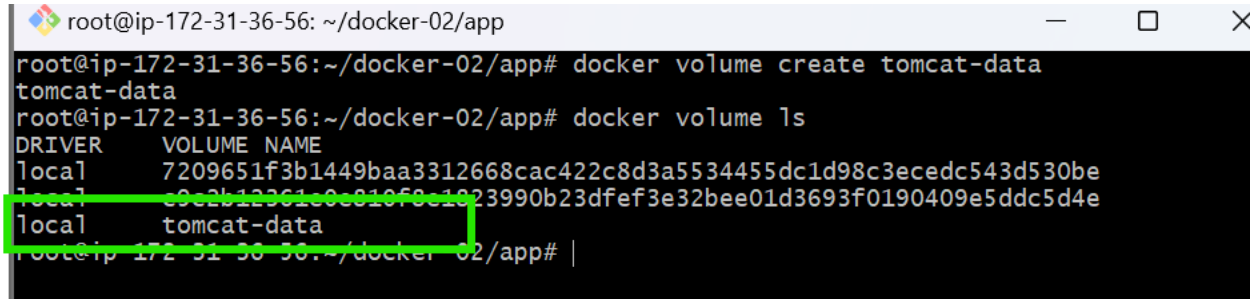
TASK 2: Create a Docker Volume and Run Tomcat on Port 8081

Step 1: Create a volume

docker volume create tomcat-data

Verify the volume:

docker volume ls

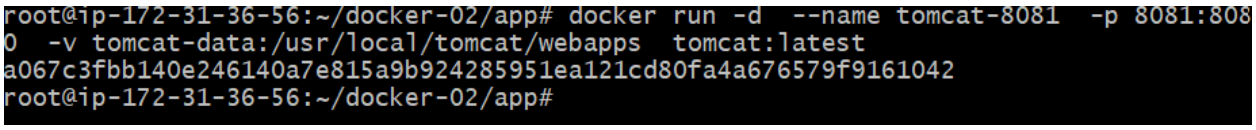


Step 2: Run Tomcat with the volume

docker run -d --name tomcat-8081 -p 8081:8080 -v tomcat-data:/usr/local/tomcat/webapps tomcat:latest

Here:

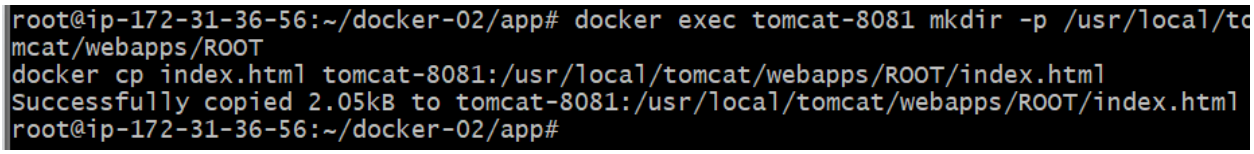
- 8081 is the host port.
- 8080 is the container port.
- tomcat-data is the Docker volume.



Step 3: Deploy the application

Run these commands from the directory containing index.html:

docker exec tomcat-8081 mkdir -p /usr/local/tomcat/webapps/ROOT
docker cp index.html tomcat-8081:/usr/local/tomcat/webapps/ROOT/index.html



Step 4: Verify

Open:

http://YOUR-EC2-PUBLIC-IP:8081/

Allow inbound TCP port 8081 in the EC2 Security Group.

Check the volume:

docker inspect tomcat-8081

```
Run 'docker --help' for more information
root@ip-172-31-36-56:~/docker-02/app# docker inspect tomcat-8081
[
    {
        "Id": "a067c3fbb140e246140a7e815a9b924285951ea121cd80fa4a676579f9161042",
        "Created": "2026-10-13T10:48:53.40436808Z",
        "Path": "catalina.sh",
        "Args": [
            "run"
        ],
        "State": {
            "Status": "running",
            "Running": true,
            "Paused": false,
            "Restarting": false,
            "OOMKilled": false,
            "Dead": false,
            "Pid": 2658,
            "ExitCode": 0,
            "Error": "",
            "StartedAt": "2026-10-13T10:48:53.503457328Z",
            "FinishedAt": "0001-01-01T00:00:00Z"
        },
        "Image": "sha256:6974cddd46f09a10a0d4ffae6cd77f9122b3cf7de8f4b7c3305f35026ac
cc81f",
        "ResolvConfPath": "/var/lib/docker/containers/a067c3fbb140e246140a7e815a9b92
4285951ea121cd80fa4a676579f9161042/resolv.conf",
        "HostnamePath": "/var/lib/docker/containers/a067c3fbb140e246140a7e815a9b9242
85951ea121cd80fa4a676579f9161042/hostname",
        "HostsPath": "/var/lib/docker/containers/a067c3fbb140e246140a7e815a9b9242859
51ea121cd80fa4a676579f9161042/hosts",
        "LogPath": "/var/lib/docker/containers/a067c3fbb140e246140a7e815a9b924285951
ea121cd80fa4a676579f9161042/a067c3fbb140e246140a7e815a9b924285951ea121cd80fa4a676579
f9161042-json.log",
        "Name": "/tomcat-8081",
        "RestartCount": 0,
        "Driver": "overlayfs",
        "Platform": "linux",
        "MountLabel": "",
        "ProcessLabel": "",
        "AppArmorProfile": "docker-default",
        "ExecIDs": null,
```

docker volume inspect tomcat-data

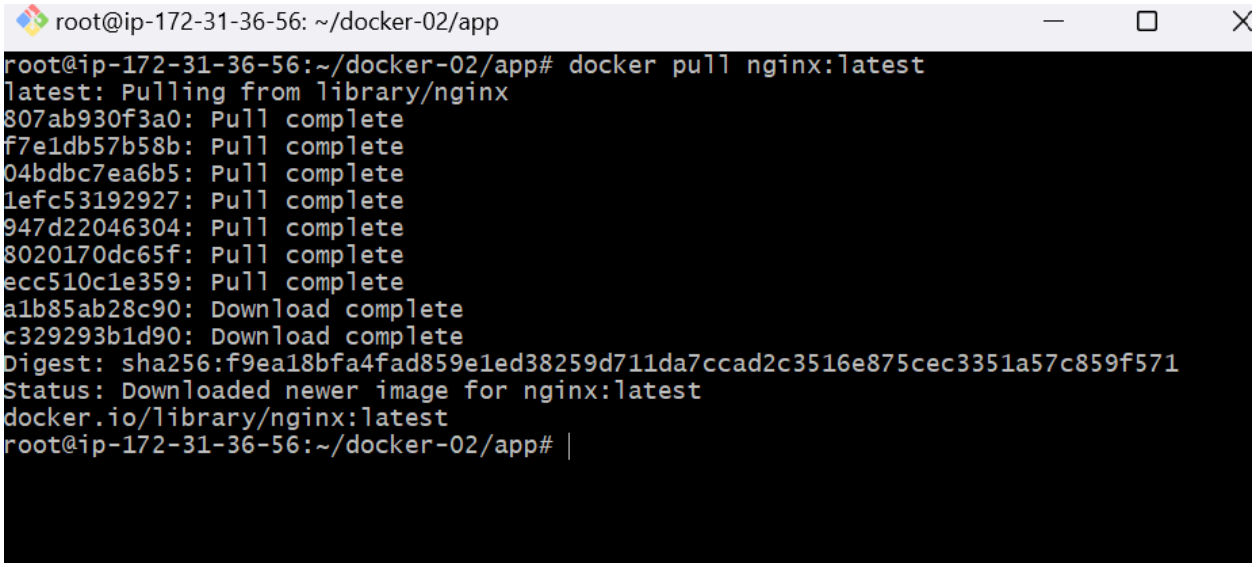
```
root@ip-172-31-36-56:~/docker-02/app# docker volume inspect tomcat-data
[
    {
        "CreatedAt": "2026-10-13T10:45:47Z",
        "Driver": "local",
        "Labels": null,
        "Mountpoint": "/var/lib/docker/volumes/tomcat-data/_data",
        "Name": "tomcat-data",
        "Options": null,
        "Scope": "local"
    }
]
root@ip-172-31-36-56:~/docker-02/app#
```

A Docker volume stores data outside the container's writable layer and can preserve data when a container is removed, provided the same volume is reused.

TASK 3: Limit the Nginx Container to 500 MB

Step 1: Pull the Nginx image

docker pull nginx:latest



Step 2: Run Nginx with a memory limit

docker run -d --name nginx-500mb -p 81:80 --memory=500m nginx:latest

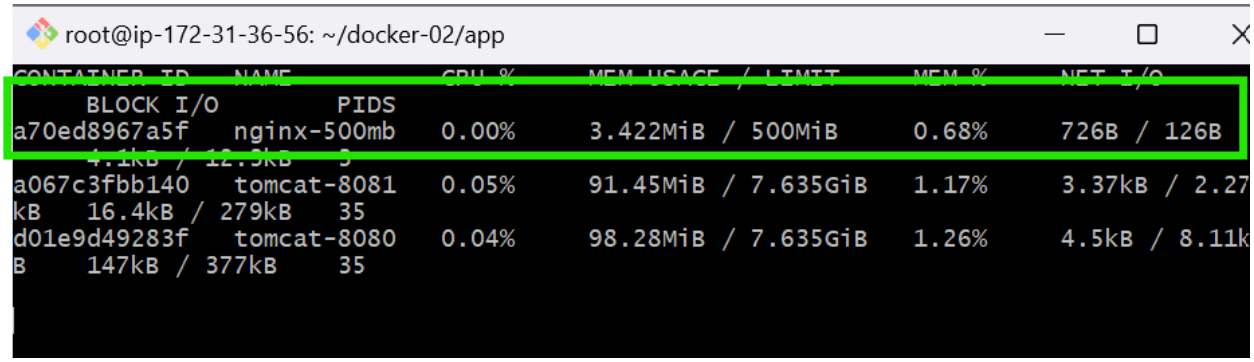
The `--memory=500m` option limits the container's memory to 500 MiB.

Step 3: Check the memory usage

docker stats

Expected output:

500MiB

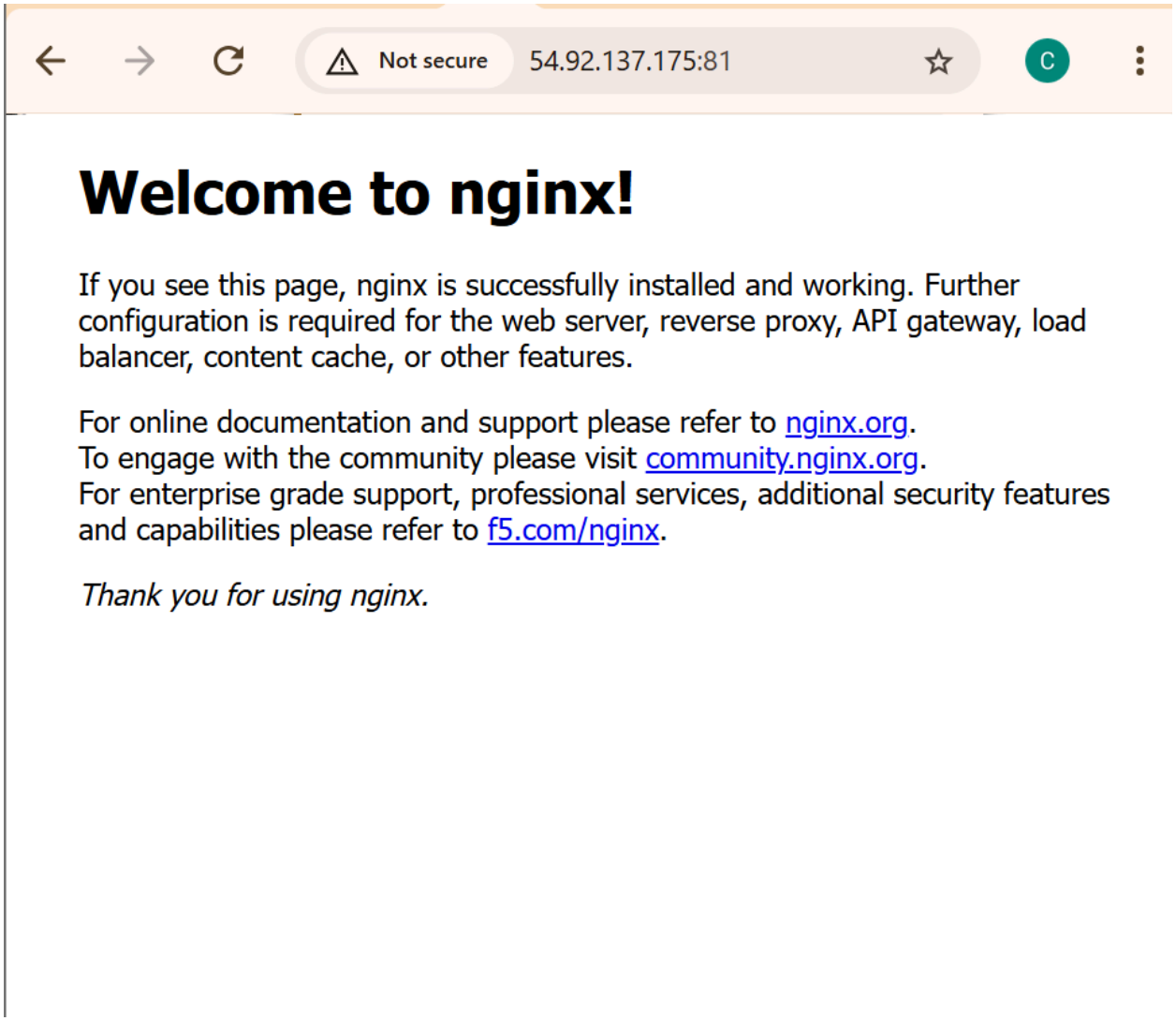


Step 4: Test Nginx

Open:

http://YOUR-EC2-PUBLIC-IP:81/

Allow inbound TCP port 81 in the EC2 Security Group.



TASK 4: Create a Dockerfile Using Amazon Linux

Step 1: Create a project directory

```
mkdir -p ~/docker-02/amazon-nginx
cd ~/docker-02/amazon-nginx
```

Step 2: Create index.html

```
nano index.html
```

Add the following content:

```
<!DOCTYPE html>
<html>
<head>
    <title>Amazon Linux Nginx</title>
</head>
<body>
    <h1>Hello from Amazon Linux Nginx!</h1>
    <p>Docker-02 assignment completed successfully.</p>
</body>
</html>
```

Save and exit.

Step 3: Create the Dockerfile

nano Dockerfile

Add the following code:

```
FROM amazonlinux:latest

MAINTAINER ch.v.karthik

RUN dnf install -y nginx && dnf clean all

COPY index.html /usr/share/nginx/html/index.html

EXPOSE 80

CMD ["nginx", "-g", "daemon off;"]
```

Explanation of Dockerfile instructions

- **FROM:** Uses Amazon Linux as the base image.
- **MAINTAINER:** Specifies the image maintainer. This is a legacy instruction.
- **RUN:** Installs Nginx.
- **COPY:** Copies index.html into the image.
- **EXPOSE:** Documents the container's listening port.
- **CMD:** Starts Nginx in the foreground.

Modern alternative to MAINTAINER:

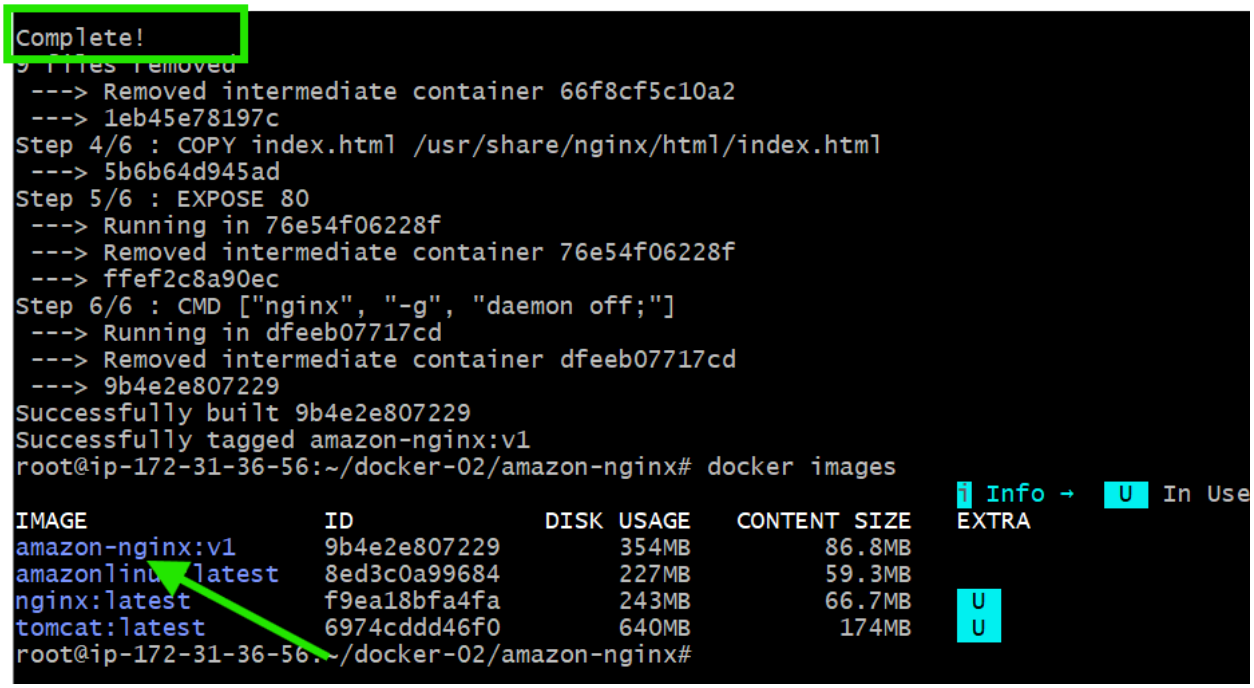
```
LABEL maintainer="ch.v.karthik"
```

Step 4: Build the image

docker build -t amazon-nginx:v1 .

Verify the image:

docker images



Step 5: Run the container

docker run -d --name amazon-nginx-container -p 8082:80 amazon-nginx:v1

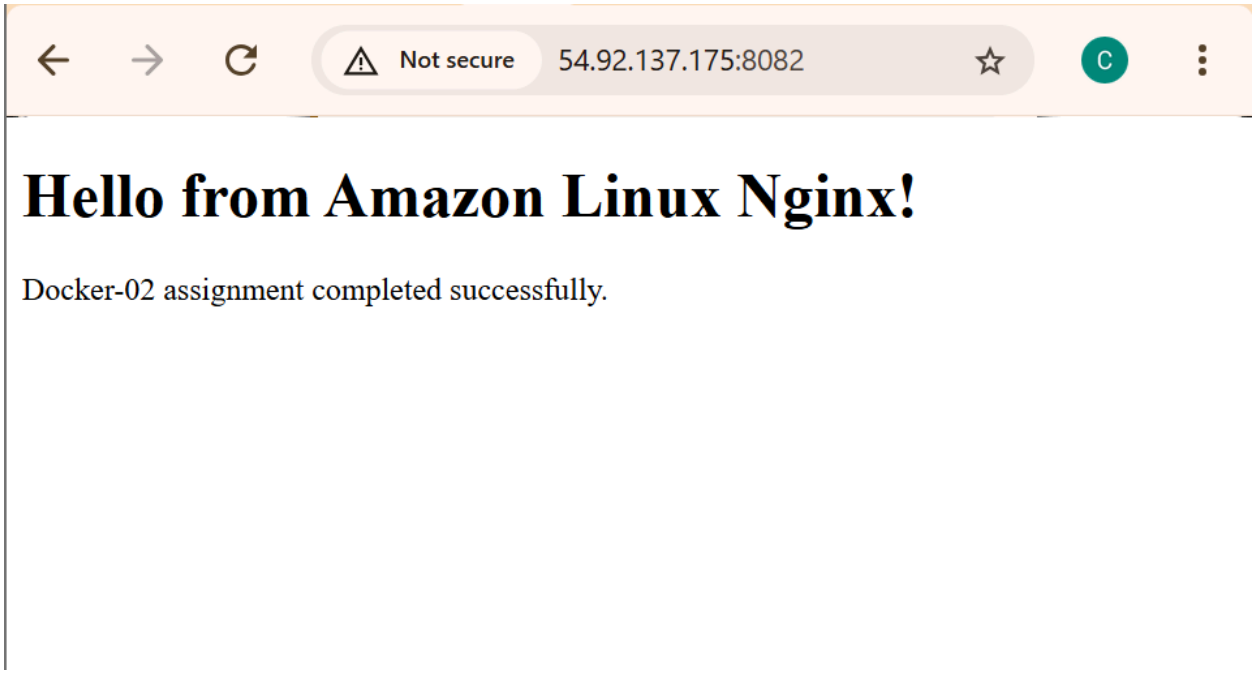
Step 6: Test the application

Open:

http://YOUR-EC2-PUBLIC-IP:8082/

Allow inbound TCP port 8082 in the EC2 Security Group.

Expected output: Hello from Amazon Linux Nginx!



TASK 5: Push the Image to Docker Hub

Prerequisite: Create an account on Docker Hub.

Step 1: Log in to Docker Hub

docker login

Use your Docker Hub username and credentials or an access token.

Step 2: Tag the image

docker tag amazon-nginx:v1 YOUR_DOCKERHUB_USERNAME/amazon-nginx:v1

Step 3: Push the image

docker push YOUR_DOCKERHUB_USERNAME/amazon-nginx:v1

```
root@ip-172-31-36-56:~/docker-02/amazon-nginx# docker tag amazon-nginx:v1 chvkarthik/amazon-nginx:v1
root@ip-172-31-36-56:~/docker-02/amazon-nginx# docker push chvkarthik/amazon-nginx:v1
The push refers to repository [docker.io/chvkarthik/amazon-nginx]
7234d0a9b6ba: Mounted from library/amazonlinux
16be687087a5: Pushed
7aa0d772b1c9: Pushed
v1: digest: sha256:9b4e2e80722965fe2229102f60435f7b32afb6247a2b8f9270091ccf783e0ffa size: 971
```

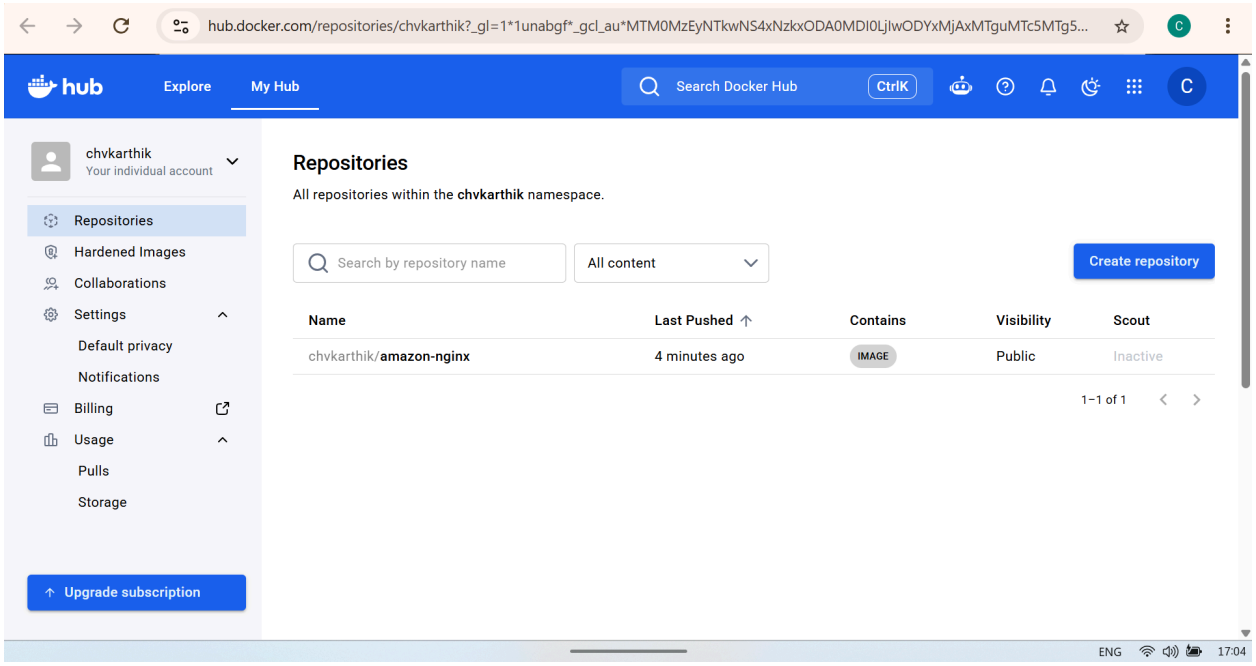
Step 4: Verify

Open:

https://hub.docker.com/

Check your repositories for:

YOUR_DOCKERHUB_USERNAME/amazon-nginx:v1



0

TASK 6: Push the Image to AWS ECR

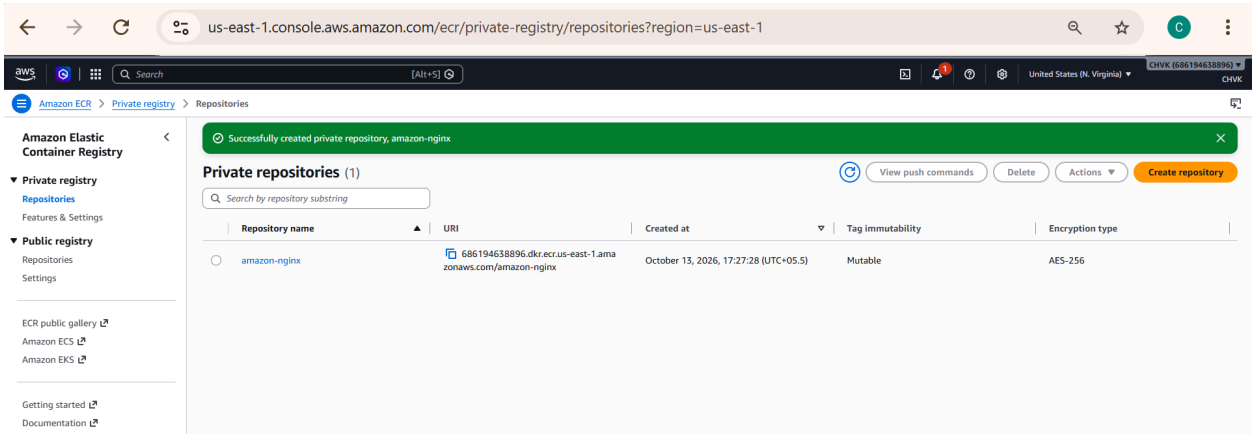
Amazon Elastic Container Registry (ECR) is an AWS service used to store Docker container images.

Step 1: Create an ECR repository

1. Open https://console.aws.amazon.com/ecr/
2. Select your required AWS Region.
3. Open Private repositories.



4. Click Create repository.
5. Enter the repository name:
   `amazon-nginx`
6. Keep the repository private.
7. Click Create repository.



Copy your AWS Account ID and Region.

Step 2: Install AWS CLI

Execute on Ubuntu:

```
sudo apt update
sudo apt install awscli -y
```

Verify:

aws --version

```
root@ip-172-31-36-56:~/docker-02/amazon-nginx# aws --version
aws-cli/2.37.13 Python/3.14.6 Linux/6.17.0-1017-aws script-exe/x86_64.ubuntu.24
root@ip-172-31-36-56:~/docker-02/amazon-nginx#
```

Step 3: Configure AWS credentials

If using an IAM user with appropriate ECR permissions:

If the EC2 instance has an IAM role with the necessary permissions, use that role instead of storing access keys.

Step 4: Log in to AWS ECR

Replace `YOUR_AWS_ACCOUNT_ID` and `YOUR_REGION` with your actual values.

aws ecr get-login-password --region YOUR_REGION | docker login --username AWS --password-stdin YOUR_AWS_ACCOUNT_ID.dkr.ecr.YOUR_REGION.amazonaws.com

```
root@ip-172-31-36-56:~/docker-02/amazon-nginx# aws sts get-caller-identity
{
    "UserId": "AROAZ7RDDIAYG6TFMEJ3V:i-0173134df62f99465",
    "Account": "686194638896",
    "Arn": "arn:aws:sts::686194638896:assumed-role/EC2-ECR-push-role/i-0173134df62f99465"
}
root@ip-172-31-36-56:~/docker-02/amazon-nginx# aws ecr get-login-password --region us-east-1 | docker login --username AWS --password-stdin 686194638896.dkr.ecr.us-east-1.
mazonaws.com

WARNING! Your credentials are stored unencrypted in '/root/.docker/config.json'.
Configure a credential helper to remove this warning. See
https://docs.docker.com/go/credential-store/

Login Succeeded
root@ip-172-31-36-56:~/docker-02/amazon-nginx#
```

Example registry format:

123456789012.dkr.ecr.ap-south-2.amazonaws.com

The account ID above is only an example.

Step 5: Tag the image for ECR

docker tag amazon-nginx:v1 YOUR_AWS_ACCOUNT_ID.dkr.ecr.YOUR_REGION.amazonaws.com/amazon-nginx:v1

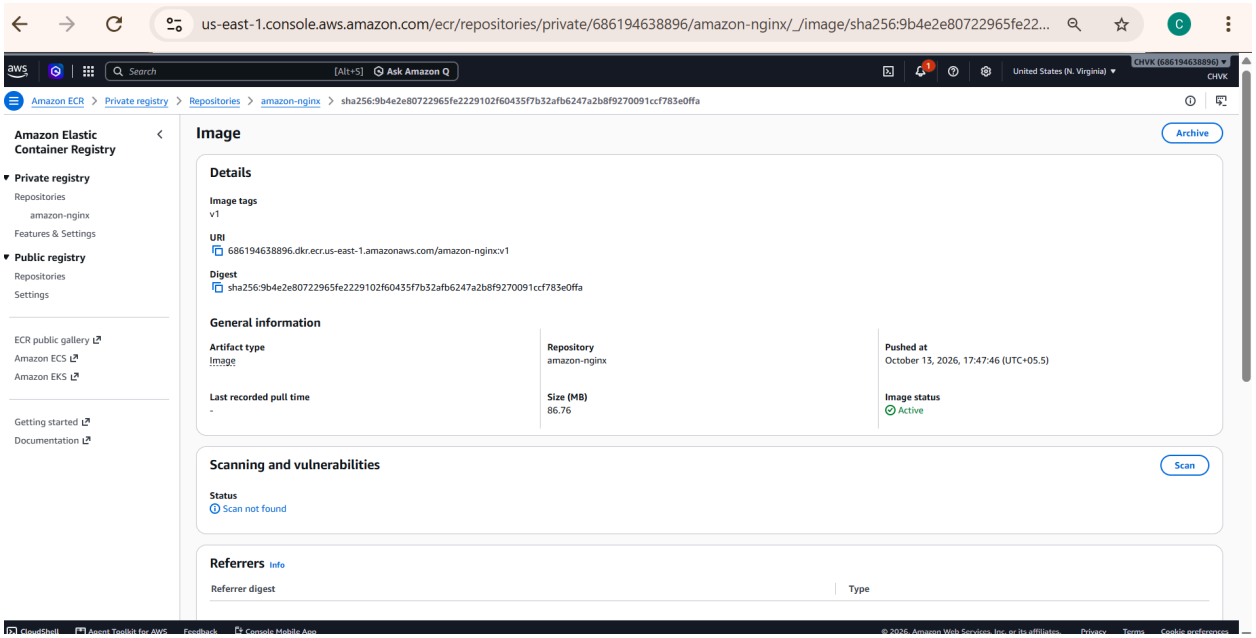
Step 6: Push the image

docker push YOUR_AWS_ACCOUNT_ID.dkr.ecr.YOUR_REGION.amazonaws.com/amazon-nginx:v1

```
root@ip-172-31-36-56:~/docker-02/amazon-nginx# docker tag amazon-nginx:v1 686194638896.dkr.ecr.us-east-1.amazonaws.com/amazon-nginx:v1
root@ip-172-31-36-56:~/docker-02/amazon-nginx# docker push 686194638896.dkr.ecr.us-east-1.amazonaws.com/amazon-nginx:v1
The push refers to repository [686194638896.dkr.ecr.us-east-1.amazonaws.com/amazon-nginx]
16be687087a5: Pushed
7234d0a9b6ba: Pushed
7aa0d772b1c9: Pushed
v1: digest: sha256:9b4e2e80722965fe2229102f60435f7b32afb6247a2b8f9270091ccf783e0ffa size: 971
root@ip-172-31-36-56:~/docker-02/amazon-nginx#
```

Step 7: Verify the image

1. Open the AWS ECR Console.
2. Open the amazon-nginx repository.
3. Select the Images tab.
4. Verify that the v1 image tag is available.



The IAM identity needs the appropriate permissions to authenticate and push images to ECR.

FINAL VERIFICATION CHECKLIST

- Tomcat container running on port 8080.
- Sample application deployed to Tomcat 8080.
- Docker volume tomcat-data created.
- Tomcat container running on port 8081 with volume.
- Nginx container limited to 500 MB.
- Amazon Linux Dockerfile created.
- Custom Docker image built successfully.
- Custom Nginx application tested on port 8082.
- Image pushed to Docker Hub.
- Image pushed to AWS ECR.

TROUBLESHOOTING

If the application does not open in your browser:

1. Check running containers:

docker ps

2. Check container logs:

docker logs tomcat-8080
docker logs tomcat-8081
docker logs nginx-500mb
docker logs amazon-nginx-container

3. Check the EC2 Security Group inbound rules.

4. Verify that the required host port is mapped correctly.

5. Confirm that the application file exists inside the container.

Conclusion

In this assignment, we created Tomcat containers, deployed a sample web application, created and used a Docker volume, limited Nginx memory, built a custom Docker image using Amazon Linux, and pushed the image to Docker Hub and AWS ECR.